动态规划自学讲义

第一册 从零建立模型

给已经学过数组和循环 正在巩固递归的你

这套讲义要帮助你学会一件事：面对很多种选择，判断哪些过去必须记住，哪些过去可以合并，然后有根据地算出答案。我们从纸上走楼梯开始，逐步走向提高组的树、集合、数位与优化。你不必先把递归学得很熟，才能开始第一册。

每次读一个小节，先完成“先停下来想”，再读推导。例题是用来学习方法的；第三册的独立练习要合上讲义做。讲义给状态、推理和局部代码，不提供整题可提交程序。输入、变量、循环、自测和最后提交由你完成。

妈妈可以检查你是否讲清“这一格是什么、为什么这样合并、有没有漏掉或重复”。程序是否正确，还需要实际运行和小规模核验。看懂、自己做出、隔天还会、换题会用，是四种不同的进展。

这套讲义怎样服务考试

按 2026 年 10 月 12 日核对的官方资料，GESP 六级涉及简单一维 DP 和简单背包，七级进一步涉及二维 DP 及最值优化，八级还要求更综合的效率分析与优化。五级的递归、分治、贪心、复杂度等是本书的重要前置知识。本书的章节顺序是教学安排，不是各级考试的官方题单。[S1]

提高组的 DP 学习还要进入多维、树形、状态压缩和常用优化等方向。第二册围绕这些能力组织，数位、概率期望和几类优化作为拓展与综合训练。会做 DP 不能替代 GESP 或 CSP-S 的数论、图论、数据结构、字符串、阅读程序与考场训练。[S2]

本书不承诺读完就能通过考试。我们把目标落实成可检查的作品：手推表、状态定义、正确性理由、独立程序、边界测试、隔天重做和陌生题迁移。

本册目录

阅读路线和使用方法 3
第 1 章 用一张小纸条记住过去 4
第 2 章 状态是决定未来所需的信息 5
第 3 章 一张表为什么算得对 6
第 4 章 递归 搜索和记忆化怎样连起来 8
第 5 章 最少硬币和不能盲信的贪心 10
第 6 章 网格把一维扩展成二维 11
第 7 章 结尾在哪里会改变状态 12
第 8 章 零一背包先把物品这一维看清 13
第 9 章 背包家族来自使用次数和计数对象 15
第 10 章 两条序列的关系 16
第 11 章 把模型写成可靠的程序 17
本册资料说明 18

目录可点击跳转。打印阅读时可按章选择页码。

阅读路线和使用方法

怎样读一节课

先读题意，画出一个小例子。遇到“先停下来想”，盖住后文，写一句自己的想法；没有想好也可以留下草稿。再读讲解，找出与你原来想法不同的地方。读到“独立任务”便停下，先完成任务，不继续向后翻找公式。

一次学习可以只推进一个完整问题。建议把讲解、独立写码、调试分成不同的学习段，每段约二十五到四十分钟，按当天精神调整。这是起始安排，不是必须完成的页数。明显疲劳时留下状态定义和卡点，下一次从题意重来。

学习阶段	阅读范围	进入下一阶段前要拿出的证据
起步	第一册第 1 至 4 章	能画选择树 解释同一问题为什么只需算一次
基础	第一册第 5 至 11 章	能独立完成线性 网格 背包和序列模型
提高	第二册第 12 至 20 章	能根据依赖设计维度 顺序和边界
优化	第二册第 21 至 24 章	先写慢方法 再证明优化没有改变答案
迁移	第三册阶段验收	无模型标签做题 留下独立建模与调试记录

递归、DFS、树图、二进制或前缀和遇到陌生处，先补对应章节的前置小课。不要靠背后面的公式跳过它。第二册的学习跨度会明显大于第一册，不要求在同一学期全部做完。

开始前把符号翻译成人话

min 表示挑最小，max 表示挑最大。f 只是表的名字，f[3]读作这张表中编号 3 的格子。f[i][j]用两个下标定位一格，例如行与列。i、j、n 等字母没有固定魔法含义，每次都要在题目里重新说明。

公式中的等号描述一个格子应当等于什么。C++中的赋值语句把右边算出的值写入左边变量；等于比较则写两个等号。正 INF 与负 INF 是我们约定的不可达标记，不是可以随意参与加减的真正无穷大。

下文的关系式采用数组下标和程序式表达，目的是方便从纸上模型过渡到代码。复杂度文字里的 2^n 表示 2 的 n 次方；C++中的^却是按位异或，不能用来求乘方。第一次遇到 O(n)、二进制或概率，可以先阅读紧邻的说明，再用小例子练习，不需要先会所有符号才开始。

第 1 章 用一张小纸条记住过去

1.1 先知道我们在解决什么

有四级楼梯。你从第 0 级出发，每次向上走一级或两级，必须恰好到第 4 级。问有多少种走法。走法按每次步长的先后顺序区分，先走 1 再走 2，与先走 2 再走 1 不同。这里问数量，不问最少走几步。

先停下来想：不用公式，把四级楼梯的全部走法有顺序地写出来。你准备怎样保证没有漏掉，也没有重复？把“第一步是 1”和“第一步是 2”分两列，或者把走法画成分叉图。

逐条列举可以得到 1111、112、121、211、22。把小题列完很重要，它给以后写的程序一个可信的检查结果。但如果有一百级，逐条写出所有走法就会越来越慢。

1.2 从最后一步往前看

站在第 4 级之前，最后一步只能来自第 3 级或第 2 级。所有到第 3 级的走法，末尾补一个 1；所有到第 2 级的走法，末尾补一个 2。这两类不会重叠，因为最后一步不同，而且没有第三种合法最后一步。

于是，我们需要的不是每条路线的全文，而是“到某一级有多少条路线”。先记小问题的答案，再拼成大问题的答案。这就是第一次接触动态规划，英文简称 DP。

```
ways[i] = ways[i-1] + ways[i-2]
```

这里方括号表示数组中的一格。ways[4]读作“到第 4 级的走法数”，不是“第 4 条走法”。i 是楼梯的位置，ways[i]是一个数量；它们的职责不同。

楼梯位置 i	0	1	2	3	4
到这里的走法数	1	1	2	3	5

第 0 级为什么记 1？因为“已经在起点，什么也不走”有一种空走法。它让从第 0 级直接跨两级这件事得到正确计数。它不是“走了一步”。第 1 级只能走 1，所以是 1。若统一让 i 从 1 开始，则只在 i>=2 时使用 i-2 这一项；绝不能访问负下标。

1.3 一份变量登记表

变量	现实含义	初值	什么时候变化
n	目标楼梯编号	输入得到	本题中不变
i	当前准备求的位置	按循环安排	一格算完后往后走
ways[i]	恰好到第 i 级的走法数	先明确 0 和 1	两个来源准备好后写入

把这张表复述给妈妈：她不需要知道数组的语法，也应能指出“位置”和“数量”不能互换。若输出 n，输出的是楼梯的高度；若输出 ways[n]，才是题目要的走法数。

1.4 独立任务

把规则改成每次走 1 级或 3 级。先列出到第 4 级的全部走法，再自己写出“最后一步来自哪里”和新的递推关系。特别检查第 0、1、2 级。先不要写代码，拿这张纸解释为什么旧公式不能原样使用。核验见第四册第 3 章。

第 2 章 状态是决定未来所需的信息

2.1 为什么可以忘记一条路线的细节

两个人都到了第 5 级。一个一路走 1，另一个混着走 1 和 2。如果后面的规则只看位置，他们面对的下一步选择相同。对于“后面怎么走”这个问题，过去的完整路线可以合并成同一个位置。

状态就是这份决定未来所需的信息。状态的名字可以叫 i，表里保存这个状态对应的答案。状态不是随便写几个下标；它是在回答“后面的选择还会受哪些过去的信息影响”。

先停下来想：现在增加规则，不能连续两次走 2 级。两个人都在第 5 级，一个最后一步是 1，一个最后一步是 2。他们下一步能做的事相同吗？只记位置还够吗？

这次不够。我们必须知道上一步是不是 2。可以把状态写成位置 i 与标记 s，s=0 表示上一步不是 2，s=1 表示上一步是 2。到了同一位置而标记不同，是两个不同状态。少记这个标记，会把合法未来不同的路线混在一起。

$$f[i][0] = f[i-1][0] + f[i-1][1]$$

$$f[i][1] = f[i-2][0] \quad \text{当 } i >= 2$$

空走法设 f[0][0]=1、f[0][1]=0，尚未走过 2。计算时对所有不存在的位置跳过来源。到第 n 级的总数是两个标记的和。第 4 级原来的 22 现在不合法，这个小例子能区分“记上一步”和“不记上一步”两种模型。

2.2 哪些历史可以合并

对于计数，同一状态的多条历史仍然是不同方案，所以把数量相加。对于求最大收益，若两条历史未来选择完全相同，且更高的当前收益不会造成额外限制，就可以只保留收益更高的一条。对于求最小费用，同样条件下只保留更便宜的一条。

如果历史 A 更便宜却耗光了一张优惠券，历史 B 稍贵却还保留优惠券，它们的未来未必相同。只记“当前位置和当前最小费用”可能丢掉后来更好的 B。优惠券是否可用也应进入状态。

这就是“无后效性”要表达的意思：状态定义好以后，未来不再需要状态之外的旧信息。它不表示题目完全没有历史影响；有影响的历史要被你放进状态。

2.3 检查状态是否足够

找两条被你写进同一个状态的不同历史，问它们未来的合法选择是否一样。若不一样，就给状态补上能区分它们的维度。若一样，再问保留的那个数是否足以比较或统计这些历史。这个方法比背“DP 五步”更能帮助你发现缺失条件。

不要见到一个条件就盲目加一维。如果“到第 i 天”已经决定星期几，就不必再单独记星期几。如果某件事能从已有下标唯一算出，也不必重复存。

2.4 独立任务

你在一条路上走，每次走 1 格或 2 格，途中最多使用一次“跳 3 格”的机会。请设计一张小纸条，让别人只看纸条就知道后面允许哪些走法。写状态含义，给出两条证明“只记位置不够”的历史；本轮不必写递推式。

第 3 章 一张表为什么算得对

3.1 给每一格写一份合同

学习新模型时，先写完整的一句话。例如“f[i]表示恰好到第 i 级的走法数”。“前 i 个”“恰好”“至多”“必须选”“以 i 结尾”都是合同里的条件。改变这些词，就可能改变初值、转移和最终输出位置。

做 DP 时常用六件事来检查：状态含义、合法来源、合并方式、起点与不可达状态、计算顺序、答案位置。它们不是填空模板。每一项都要能从题意解释出来。

问题类型	同一状态的来源怎样合并	没有合法来源时
有多少方案	不同且不重叠的来源相加	0
是否可行	任一来源可行就可行	false
最小费用	取来源代价的最小值	INF 表示不可达
最大收益	取来源收益的最大值	负 INF 表示不可达

空状态的答案要另行确定。计数常见空方案为 1；求最小费用常见起点为 0。这不是把整张表都初始化成 1 或 0 的理由。尤其“恰好达到某容量”时，其他容量起初未必可达。

3.2 转移为什么不会漏也不会多

楼梯题按最后一步分成两类。要证明公式正确，先说明任何合法走法一定在其中一类里，这是不漏；再说明两类不会代表同一条完整走法，这是不重。最后说明把合法的小走法补上最后一步后仍然合法，这是不加入错误方案。

对于最值题，还要说明丢掉较差历史为什么安全：替换成同状态下更好的历史，不改变后面能选的动作。这叫最优子结构。它依赖你定义的状态足够完整，不能只因为题目出现“最大”或“最小”就认定成立。

3.3 为什么要从小到大填

若 ways[4]需要 ways[3]与 ways[2]，就必须先准备好它们。可以画箭头：从一个格子指向使用它的格子。如果箭头一直指向更大的编号，从小到大填写就合理。

以后遇到区间，要按长度填；遇到树，要先算孩子再算父亲；遇到有向无环图，要按依赖顺序填。循环顺序来自依赖关系，不来自“DP 都从 1 开始”的口诀。

证明整张表正确，也可以跟着这个顺序：起点正确；假设依赖的小问题都正确，本格的分类与合并保证本格正确；最后就能走到答案。这是归纳思想。开始时用自己的话解释即可，不必急着写正式数学证明。

3.4 一次填表需要多久

若一共有 n 个状态，每格最多查看 2 个来源，工作量与 n 成正比，记为 O(n)。若 n 格中的每格都检查前面的 n 个位置，总工作量大约是 n^2，记为 $O(n^2)$。空间复杂度统计同时保留多少格，而不是循环执行多少次。

判断能否运行，要把状态数、每状态转移数、常数、时限和内存一起看。十万的平方是百亿，通常不适合直接双重枚举；具体能跑多少次不能只用一个固定口号代替测量。

3.5 独立任务

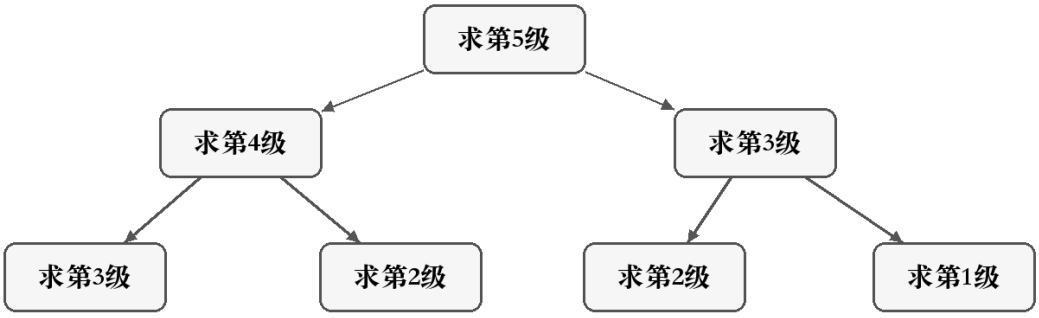
有人说：“我从第 n 级向第 1 级填楼梯表，反正公式没改，答案也一样。”请用第 4 级解释第一次读到哪个尚未准备好的数，再画出合法的依赖顺序。完成后检查自己的上一章状态表是否也有可靠的计算顺序。

第 4 章 递归 搜索和记忆化怎样连起来

4.1 先用问答认识递归

递归可以看作把同一种问题交给更小的自己。问“到第 4 级有几种走法”，需要先知道“到第 3 级”和“到第 2 级”。每个问题都必须有清楚的返回值，遇到第 0 级等简单情况时直接回答。

一次函数调用有自己的一份参数和局部变量。外层调用等内层回答后再继续，不是把全世界的 i 一起改掉。手推时画几张纸片，每张写“我正在求哪个 n、我等哪个结果”，比在脑中追踪十层调用容易。



重复出现的相同问题 可以共享已经算出的答案

深度优先搜索 DFS 是一种遍历选择的方式：沿一个分支继续走，走完或失败再回来试其他分支。递归经常用于实现 DFS，但递归不等于 DFS，DFS 也可以用显式栈实现。此处先理解“沿分支探索”和“返回后继续”。

4.2 展开和返回是两个方向

以“求 1 到 n 的和”为例，sum(3)先停在“3 加上什么”，等待 sum(2)；sum(2)停在“2 加上什么”，等待 sum(1)。sum(1)直接回答 1，于是 sum(2)回答 3，最后 sum(3)回答 6。

调用	先知道的事	等待谁	返回的数
sum(3)	需要把 3 算进去	sum(2)	3+3=6
sum(2)	需要把 2 算进去	sum(1)	2+1=3
sum(1)	已经到边界	不再等待	1

从 3 走到 1 是在展开问题，从 1 回到 3 是在把答案带回来。外层不是消失了，而是暂时暂停。楼梯会等待两个小问题，原理仍一样。先把一条链的展开与返回说清，再追踪分叉。

4.3 重复计算发生在哪里

求第 5 级会求第 4 级和第 3 级；求第 4 级又会求第 3 级。两次“第 3 级”若状态含义完全相同，它们会得到同一答案。把算好的结果记在表里，下次直接取用，就叫记忆化。

记忆化的通用流程

求解某状态：
  如果是边界，返回边界答案
  如果这个状态已经算完，返回记录值
  依次求解所有合法且规模更小的来源
  按题意合并来源，记录结果并标记算完
  返回记录值

这是流程说明，不是可提交代码。你必须自己定义“状态”“边界”“来源”和“合并”。不能把答案是否为 0 当作是否算过：有些状态的正确答案就是 0。可以另设 done 数组，或使用一个不可能是合法答案的标记。

4.4 两种计算安排

记忆化搜索是先问大问题，需要哪个小问题才去算哪个；递推填表是先列好顺序，把需要的小问题依次算完。两者可以使用相同的状态与转移。稀疏状态适合按需计算；依赖规则清楚、状态稠密时，循环填表通常直观，也避免过深递归的栈风险。

一个函数绕一圈又要求尚未算完的自己，记忆化不会自动解决循环依赖。例如图里有 A 到 B、B 到 A，不能仅靠“结果算完后存一下”保证结束。首先要证明依赖无环，或使用适合这类循环方程、最短路等问题的方法。

楼梯题有 n 个不同状态，每个状态的来源数是常数。记忆化以后总工作量 O(n)，但递归深度仍可能 O(n)。省掉重复计算和省掉调用栈是不同的事。

4.5 前置小课和独立任务

请先手推一个函数“求 1 到 n 的和”，画出 n=3 时三个调用各自等待的结果，再说明 n 怎样变小、在哪停止。这个任务卡住就先巩固递归，不急着写记忆化。

之后把楼梯 n=5 的调用画到重复状态出现，给同一个状态涂同一种颜色。数一数“问题被问了几次”和“不同问题有几个”。最后自己选择记忆化或循环方式实现第一章的原题，测试 n=0、1、4。n=0 是我们定义下的合法最小情况。

第 5 章 最少硬币和不能盲信的贪心

5.1 同样是到达目标 这次要求不同

有面值 1、3、4 的硬币，每种都有很多枚。要恰好凑出 6 元，最少用几枚？先停下来想：每次取当前能拿的最大硬币，会得到什么？有没有更少的方案?

取 4 以后只能再取 1 和 1，一共 3 枚；取 3 和 3 只要 2 枚。局部最大的选择没有保证全局最优。DP 保留不同金额的最优答案，让最后一步的不同可能都参加比较。

定义 f[x]为恰好凑出 x 元所需的最少硬币数。最后取面值 c，就必须先凑出 x-c 元，并且再多用一枚硬币。每一种 c 都是候选来源。

$$f[x] = \min(f[x-c] + 1)$$

这个 min 只枚举 c<=x 且 x-c 可达的面值。f[0]=0；其他位置先设为 INF，表示暂时没有合法方案。计算 x 时只依赖更小金额，因此从 1 到目标金额填写。

金额 x	0	1	2	3	4	5	6
最少枚数	0	1	2	1	1	2	2

这里不能把各来源相加，因为题目要最少枚数；也不能把不可达写成 0，否则“没有办法凑成”的状态会冒充“免费就凑成了”。如果面值只有 3 和 4，金额 1 和 2 应保持不可达。

5.2 一个最优方案为什么能接在最优小方案后面

假设某个凑 6 元的最优方案最后取 3 元，前面那部分就是凑 3 元。如果前面不是凑 3 元的最省方案，换成更省的凑 3 元方案，面额不变、后面的最后一枚仍然能用，总枚数还会减少，与原来最优矛盾。

这里替换之所以成立，是每种硬币数量不受限制。如果总共只有一枚 3 元，状态只记金额就可能不知道这枚硬币是否已用，需要改变模型。最优子结构必须和题目条件一起检查。

5.3 从答案追溯一个方案

如果还要求输出拿了哪些硬币，可以在每次更新最优值时，记录这次选的最后一枚。到目标金额后，反复减去记录的面值，直到 0。并列最优可以任取一个；若题目要求字典序最小，还要额外处理并列规则。

一维压缩后的记录是否可靠要另行检查。硬币无限使用且金额严格减小，追溯可行；0/1 背包若只记录会被覆盖的容量前驱，可能重复拿到同一物品。后面会回到这个问题。

5.4 独立任务

面值只有 2 和 5，目标 7。请写出 0 到 7 的表，明确不可达格，再写“为什么从不可达格不能继续假装拿一枚就成功”。完成后换成目标 3，检查程序输出的含义。输出约定由你在编码前写清。

第 6 章 网格把一维扩展成二维

6.1 坐标是两个变化量

一个 3 行 4 列的棋盘，从左上角走到右下角，每次只能向右或向下。第 2 行第 2 列有障碍，不能进入。问有多少条合法路径。起点和终点都算格子，坐标从 1 开始。

先停下来想：到某一格前的最后一步，可能来自哪两格？有障碍的格子应当保存多少条路线？先画格子，不写数组。

定义 f[r][c]为从起点恰好走到第 r 行第 c 列的路径数。非障碍格由上边和左边进入；越界来源不存在。起点如果没有障碍，只有一条空路径。

$$f[r][c] = f[r-1][c] + f[r][c-1]$$

行与列	第 1 列	第 2 列	第 3 列	第 4 列
第 1 行	1	1	1	1
第 2 行	1	障碍 0	1	2
第 3 行	1	1	2	4

逐行从左向右填，保证上面和左面已经准备好。初始化可选择显式处理起点与边界；也可以增加一圈值为 0 的虚拟边界，再放一个虚拟入口。两种办法都能正确，但不要同时使用而重复贡献。初学先用显式起点。

若起点或终点有障碍，答案为 0。整行或整列被封住可能使后面的区域不可达。只用没有障碍的样例，无法检查这些分支。

6.2 路径最多能收集多少价值

若每格还有价值 a[r][c]，走过就取得价值，目标改成最大总和，坐标模型仍有用，但表里保存的是最优分数。起点值应是 a[1][1]；不可达格应设负 INF。到一个普通格子，取合法来源的较大分数后，加本格价值一次。

如果格子可能为负数，“边界设 0”会虚构从棋盘外免费进入的路线。只有当状态含义允许从任意处重新开始时，0 才可能代表合法选择。初始化永远从题意来。

6.3 独立任务

先实现有障碍的路径计数。用上表、1 行 1 列、起点有障碍三类数据自测。然后只写出“最大价值版本”的状态、初值和转移，不急着改程序，向妈妈说明为什么计数的加法变成取最大值。

第 7 章 结尾在哪里会改变状态

7.1 连续一段的最大和

给出整数序列-2、3、-1、4、-5。要选一段连续且非空的数，使和最大。连续表示中间不能跳过。先停下来想：最后一个数是负的时，最好的那一段一定以最后一个数结尾吗？

定义 end[i]为必须以第 i 个数结尾的非空连续段最大和。若选 a[i]，前面只有两个选择：在 a[i]重新开始；或把以 i-1 结尾的最好连续段接过来。不能接更早但不包含 i-1 的一段，否则不连续。

$$end[i] = \max(a[i], end[i-1] + a[i])$$

位置 i	1	2	3	4	5
a[i]	-2	3	-1	4	-5
end[i]	-2	3	2	6	1
截至这里的全局最好	-2	3	3	6	6

end[n]只回答“以最后一个数结尾”，所以总答案要在全部 end[i]里取最大值。全负数据如-4、-2，非空答案应为-2。把初值设 0 并允许空段，会改变题目。

7.2 严格上升但允许跳过

最长严格上升子序列 LIS 要求保持原来的先后顺序，可以跳过一些位置，选出的值必须严格变大。例如 3、1、2、2、4，最长长度是 3，可以选 1、2、4。两个相等的 2 不能同时让严格上升长度增加。

定义 len[i]为必须以 a[i]结尾的最长严格上升子序列长度。前一个选中位置 j 可以是任何 j<i 且 a[j]<a[i] 的位置。没有合适 j 时，a[i]自己构成长为 1 的序列。

$$\text{len}[i] = \max(1, \text{len}[j] + 1)$$

其中 j 只在合法前驱里枚举。共有 n 个状态，每个最多检查 n 个前驱，时间 $O(n^2)$，空间 $O(n)$。总答案仍是全部 len[i]的最大值。状态“以 i 结尾”让未来能判断是否可以接上新值。

如果只记“前 i 个数的最长长度”，就会丢掉结尾大小。同样长的两个子序列，结尾小的往往更容易延长。第二册会利用这个信息得到更快算法，但此刻先把朴素模型写对。

7.3 连续和子序列要分清

“连续”与“子序列”是题意条件，不是词语细节。连续段只能接 i-1；上升子序列可以接更早的 j，却必须检查值的大小。转移差别来自允许的动作，不能因为两题都有 max 就复制公式。

7.4 独立任务

请分别用 2、2、2 与 5、4、3 检查严格上升子序列的程序。再设计一个例子，证明最长上升子序列不一定以最后一个数结束。向妈妈解释为什么这里取最大值，却不能把不同前驱的长度直接相加。

第 8 章 零一背包先把物品这一维看清

8.1 物品只能拿一次

有三件物品，重量与价值分别为(2,3)、(3,4)、(4,5)。背包容量 5，物品不可拆分，每件最多拿一次，总重量不超过容量，求最大价值。这里“最多拿一次”叫 0/1，“不超过”表示不必装满。

先停下来想：只按价值高低挑、只按价值除以重量挑，能保证这组和所有其他数据都最优吗？先列三件物品的所有子集，删掉超重的组合。

对第 i 件物品做选择时，之前能用哪些物品很重要。定义 f[i][c]为只考虑前 i 件物品、容量上限为 c 时的最大价值。最后分成“不选第 i 件”和“选第 i 件”两类。

$$f[i][c] = \max(f[i-1][c], f[i-1][c-w[i]] + v[i])$$

第二项只有 c>=w[i]时合法。两项都来自第 i-1 层，保证第 i 件物品只使用一次。第 0 层没有物品，在“最多装到 c”且允许空集的定义下，各容量的答案都是 0。

考虑的物品	容量 0	容量 1	容量 2	容量 3	容量 4	容量 5
没有物品	0	0	0	0	0	0
加入第一件	0	0	3	3	3	3
加入第二件	0	0	3	4	4	7
加入第三件	0	0	3	4	5	7

答案是 f[n][W]。表格中 f[1][5]=3 不表示重量恰好为 5，而是容量上限为 5 时拿第一件最划算。把这句话讲明白，再讨论压缩。

8.2 为什么一维时容量倒着走

计算新一层只用上一层，可以只保留一行 dp[c]。但覆盖顺序必须保证读取 dp[c-w]时，它仍代表“未考虑当前物品”的旧值。容量从大到小处理，c-w 更小，还没有在本轮被更新，因此能保住旧层含义。

最小反例只有一件重量 2、价值 3 的物品，容量 4。若容量从小到大，更新到 c=2 后 dp[2]=3；随后 c=4 会读到刚更新的 dp[2]，又加 3，变成 6。同一件物品被用两次，违反题意。

局部代码 只展示层次与方向

```
for (int c = W; c >= w; --c) {
    dp[c] = max(dp[c], dp[c - w] + v);
}
```

这段代码假定物品的 w、v 已经读入并处在处理当前物品的一轮中，dp 此前也按“不超过”的含义初始化。它不是完整程序。写自己的程序前，在纸上标注每次读的是旧层还是新层。

8.3 恰好装满是另一个合同

如果要求总重量恰好等于 W，设 g[c]为恰好重量 c 时的最大价值。开始只有 g[0]=0，其他容量是负 INF。只有 g[c-w]可达时才能更新。最终 g[W]不可达就应按题目要求报告无解。

若还要输出选中了哪些物品，初学保留二维表，从第 n 层向前比较“不选”和“选”的来源。仅在一维表存一个会被后续覆盖的容量前驱，可能追出不合法重复物品，不能想当然地压缩方案记录。

8.4 独立任务

先独立实现二维 0/1 背包，再用同一组小数据改成一维并核对每一层。测试“一件物品容量足够放两件”、容量 0、所有物品都过重。最后将题意改为“恰好装满”，写出需要改变的初始化和可达性检查。

第 9 章 背包家族来自使用次数和计数对象

9.1 完全背包允许重复使用

完全背包里每种物品可以选任意多件，重量必须为正以保证容量依赖向前推进。选当前物品之后，还可以继续使用它。所以二维转移的“选”分支来自本层，而“不选”分支来自上一层。

f[i][c] = max(f[i-1][c], f[i][c-w[i]] + v[i])

一维压缩时容量从小到大，故意读取本轮更新后的较小容量。这与 0/1 的不同不是一条待背口诀，而是“当前物品还可不可以再用”的结果。若出现零重量正价值且无限次，最大价值无上界，应首先检查题目条件。

9.2 多重与分组先从清楚的慢方法开始

多重背包中第 i 种最多 s[i]件。最直接是在旧层枚举使用 k 件，0<=k<=s[i]且 k*w[i]<=c，取 f[i-1][c-k*w]+k*v 的最大值。时间可达 O(W 乘各数量之和)。每个候选都读旧层，避免超过上限。

二进制拆分把数量例如 13 拆成 1、2、4、6 四包，包的重量价值分别乘相应数量，随后按 0/1 处理。任意 0 至 13 件都能由某些包表示，所以求最值时可用。由于某些数量可能有多种包的组合表示，不能未经证明把这种拆法用于“原物品选择方案数”。

分组背包要求每组最多选一件。对同组的所有候选，必须基于进入这一组之前的旧层计算，再统一得到新层。若边枚举本组物品边读已更新结果，可能同组选了多件。用 old 和 next 两个数组最容易看清；若每组必须选一件，不能直接保留“不选本组”分支。

9.3 计数时先问怎样算同一种方案

面值 1 和 2，凑出 3。若 1+2 与 2+1 算同一种，是无序组合，共有{1,1,1}和{1,2}。若先后顺序算不同，是有序序列，共有 111、12、21。目标都是计数，问题却不同。

组合数的常见实现是先枚举硬币种类，再从小金额到大金额更新；它按“可用前几种硬币”分层，让每个组合只有一种归属。序列数则先枚举目标金额，再枚举最后一枚硬币；它按最后一步分类，因此 12 与 21 分别计入。

这些结论依赖这里的模型：正面值、每种无限量、不同种类按面值区分。若输入有重复面值，或者物品按编号区分，必须先明确题目怎样定义不同方案，不能直接套循环顺序。

9.4 独立任务

请给“面值 1 和 2、目标 4”的组合与序列各列一张清单，再画出各自最后一次更新的来源。只有清单与程序一致，才可以说自己理解了循环顺序。另找一个同组两件物品的例子，证明分组背包不能组内连续叠加。

第 10 章 两条序列的关系

10.1 公共子序列为何需要两个进度

给字符串 A=ABC、B=AC，求最长公共子序列 LCS 长度。子序列可以跳过字符，但必须保持顺序。两边都要描述“已经考虑到哪里”，所以需要两个前缀长度。

定义 f[i][j]为 A 的前 i 个字符与 B 的前 j 个字符的最长公共子序列长度。空前缀的答案是 0。为便于解释，这里字符编号从 1 开始；若 C++字符串从 0 开始，A 的第 i 个字符实际写 A[i-1]。

当末尾字符相等，可以让它们配成一对，再接在两个更短前缀的公共子序列后面；当末尾不同，它们不可能作为同一对匹配，至少有一个末尾不被选中。为避免初学者漏分支，可以始终比较跳过任一末尾，并在相等时再比较配对。

f[i][j] = max(f[i-1][j], f[i][j-1])

若 A[i] == B[j] 再与 f[i-1][j-1] + 1 比较

A 前缀与 B 前缀	空	A	AC
空	0	0	0
A	0	1	1
AB	0	1	1
ABC	0	1	2

从左上到右下填写，时间 O(nm)。如果要求方案，可从末尾倒着追踪使用了哪个来源。字符相等时也可以使用常见简化公式 f[i-1][j-1]+1；上面的多候选写法更直接展示允许忽略末尾。

10.2 编辑距离换了表中保存的东西

把 A 变成 B，允许插入、删除或替换一个字符，每次代价 1。定义 d[i][j]为把 A 前 i 个字符变成 B 前 j 个字符的最少操作数。d[i][0]=i、d[0][j]=j，这两条边表示删光或从空串插入。

最后一次对齐有三种情形：删除 A 末尾，来自 d[i-1][j]+1；插入 B 末尾，来自 d[i][j-1]+1；配对两边末尾，来自 d[i-1][j-1]，相等不加代价，不同加 1。取最小值。kit 变成 sit 只需替换首字母一次。

LCS 与编辑距离表面上都有二维表，却保存不同对象、使用不同代价。不要因为表的样子相似就混用初始化。尤其 LCS 的空前缀是 0，编辑距离的边界是一串连续的插入或删除成本。

10.3 空间压缩前先认出左上角

只保留上一行与当前行，空间可降为 O(m)。若进一步只保留一行，还要单独保存被覆盖前的左上角。没有讲清每个来源来自哪一行之前，不必急着使用最省空间的版本。

10.4 独立任务

自己填出 A=ABA、B=BA 的 LCS 表，解释最终答案在哪个格。然后只手推 A 为空串、B=ABC 时的编辑距离边界，说明为什么整行不能全设 0。

第 11 章 把模型写成可靠的程序

11.1 从中文状态到数组

先写一行注释：“dp[i][j]表示……”。列出每个下标的有效范围，决定分配多大的数组。再写初始化，随后按依赖实现转移，最后单独核对输出位置。这一顺序能减少“程序写完才想变量是什么意思”的混乱。

你可以先写清楚但稍慢的版本，打印小数据的每一层。看到第一格与手推不一致时，检查那一格的来源和初值。不要因为最终答案不对就立刻重写整题，也不要同时修改状态、边界和循环三个部分。

11.2 数值范围是模型的一部分

楼梯计数会增长得很快。本书约定 ways[0]=ways[1]=1，ways[91]=7540113804746346429，还在通常 64 位有符号 long long 范围内；ways[92]=12200160415121876738，已经超出。递推式正确仍然可能算错，因为存放答案的容器不够大。

若题目要求精确大整数，就需要高精度；若明确要求对 M 取模，可以在计数加法中取模。不能为了避免溢出擅自取模。求最小值或最大值时，通常不能先对各候选取模再比较，因为取模会改变大小关系。

long long 类型的变量接收结果，不代表右边运算已经自动用 long long。例如两个 int 先相乘可能已溢出，需在乘法发生前提升类型。INF 也要留出加法余量，并且先检查来源可达，再相加，避免对哨兵继续运算。

11.3 三类测试加一次小规模核验

第一类是关键区分例：专门让错误思路失败，例如 0/1 背包只有一件物品、容量却能装两件。第二类是边界：全负、不可达、相等元素、起点障碍等。第三类是最小规模：空前缀、单个元素、容量 0，前提是题目允许这些输入。

小规模核验是写一个很小规模能列举所有方案的笨办法，再与 DP 比较许多小输入。笨办法可以慢，但要独立按题意实现，不能把 DP 的同一条错误公式再抄一遍。两者不同后，保留最小失败样例，找第一处分歧。

跑过许多数据提高信心，不能代替正确性理由。正确性理由也不能替代检查实现。两者配合，才知道“想法对”和“程序真按想法做了”。

11.4 第一册验收

合上讲义，从第三册完成 A 组。妈妈只读题，不提醒题型。你要独立留下：状态的一整句话、一个手推例子、转移的来源分类、初值、顺序、复杂度、程序和三类测试。隔一到三天换一题重做。

如果你能跟着公式填表，却说不清为什么旧历史可以合并，回第 2 章；若模型正确但程序错，优先对照变量与每层含义，不必把所有理论重学。记录实际提示，不把“提示后完成”当作独立通过。

本册资料说明

[S1] 中国计算机学会 GESP 认证大纲页面及其 C++与 Python 一至八级附件。仅用于范围核对，讲解、例题与练习为本讲义原创教学编排。核对日期 2026 年 10 月 12 日。

https://gesp.ccf.org.cn/101/1008/10012.html

https://gesp.ccf.org.cn/101/attach/1725701770182688.pdf

[S2] 中国计算机学会 NOI 大纲 2025 年修订版发布页。提高组章节为教学路线，不等同于官方逐题要求或得分保证。

https://www.noi.cn/xw/2025-04-18/841584.shtml

https://www.noi.cn/upload/resources/file/2025/04/18/NOI_Syllabus_Edition_2025.pdf